

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Операционные среды и системное программирование»

ОТЧЁТ
к лабораторной работе №1
на тему
«СКРИПТЫ SHELL»
БГУИР 6-05-0612-02 23

Выполнил студент группы 353503
СЕБЕЛЕВ Дмитрий Юрьевич

(дата, подпись студента)

Проверил ассистент каф. информатики
ГРИЦЕНКО Никита Юрьевич

(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1	Постановка задачи	3
2	Описание функционала клиента	4
2.1	Архитектура скрипта	4
2.2	Обработка аргументов командной строки	4
2.3	Автоматическое определение языка	4
2.4	Создание paste	4
2.5	Скачивание paste	5
	Вывод	6
	Список использованных источников	7
	Приложение А (обязательное) Листинг программного кода	8

1 ПОСТАНОВКА ЗАДАЧИ

Разработать shell-скрипт – клиент для сервиса *pastebin* (*paste.rs*). Скрипт должен поддерживать создание текстовой записи из *stdin* или файла с автоматическим или ручным выбором языка подсветки синтаксиса. Необходимо обеспечить возврат идентификатора созданной записи. Также требуется реализовать скачивание записи по *id* с выводом на консоль или сохранением в файл. Создание и скачивание должны выполняться одним скриптом.

2 ОПИСАНИЕ ФУНКЦИОНАЛА КЛИЕНТА

2.1 Архитектура скрипта

Скрипт реализован на языке *Shell* с использованием строгого режима выполнения *set -euo pipefail*, обеспечивающего немедленное завершение при ошибках. Архитектура построена на модульном принципе с разделением функциональности на блоки обработки аргументов, определения языка программирования, создания и скачивания записей. Для управления временными файлами применяется команда *trap*, гарантирующая их удаление при любом способе завершения скрипта.

2.2 Обработка аргументов командной строки

Обработка параметров реализована с помощью команды *getopts*. Скрипт поддерживает флаги: *-f* для указания входного файла, *-l* для задания языка программирования, *-t* для выбора типа контента (*text* или *code*), *-o* для выходного файла при скачивании, *-h* для вывода справки. Параметр *-t* определяет форматирование: при значении *code* содержимое оборачивается в *markdown*-блок с указанием языка. После обработки опций выполняется сдвиг позиционных параметров через *shift*, что позволяет использовать оставшийся аргумент как путь к файлу или идентификатор записи.

2.3 Автоматическое определение языка

Функция *detect_lang* определяет язык программирования по расширению файла с помощью конструкции *case*. Поддерживаются распространённые языки: *Python*, *Shell*, *JavaScript*, *TypeScript*, *C/C++*, *Java*, *Go*, *Rust*, *Ruby*, *PHP*, *HTML*, *CSS*, *JSON*, *Lua*, *Markdown*. При несовпадении расширения с известными шаблонами функция возвращает пустую строку, и контент отправляется без указания языка.

2.4 Создание paste

При создании записи язык определяется автоматически по расширению файла или задаётся явно через параметр *-l*. Содержимое подготавливается во временном файле, созданном командой *mktemp*. Если указан тип *code*, содержимое оборачивается в *markdown*-блок с указанием языка. Отправка выполняется командой *curl -sS --data-binary* на сервер *paste.rs*. Сервер возвращает *URL*, из которого извлекается идентификатор командами *basename* и выводится в *stdout*.

2.5 Скачивание paste

Скачивание выполняется при передаче идентификатора или *URL* в качестве аргумента. Скрипт определяет формат аргумента регулярным выражением и формирует полный *URL* при необходимости. При указании параметра *-o* содержимое сохраняется в файл с автоматическим удалением *markdown*-ограждения кода, если расширение выходного файла соответствует языку в блоке. Для обработки используются команды *sed* и *grep*. Без параметра *-o* содержимое выводится в *stdout* командой *curl -sSf*.

ВЫВОД

В ходе выполнения лабораторной работы был разработан клиент для сервиса *paste.rs* на языке *Shell*. Скрипт реализует создание текстовых записей из *stdin* или файла с автоматическим определением языка программирования, возврат идентификатора созданной записи, скачивание по *id* с возможностью сохранения в файл.

Разработанное приложение демонстрирует использование ключевых элементов *shell*-скриптов: строгий режим выполнения, обработка аргументов командной строки через *getopts*, работа с временными файлами и командами *curl*, *sed*, *grep*. Полученный опыт применим для решения задач автоматизации и системного программирования.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Bash Reference Manual [Электронный ресурс]. – Режим доступа: <https://www.gnu.org/software/bash/manual/bash.html>. – Дата доступа: 12.02.2026.
- [2] curl - command line tool and library for transferring data with URLs [Электронный ресурс]. – Режим доступа: <https://curl.se/docs/manpage.html>. – Дата доступа: 12.02.2026.
- [3] paste.rs - simple pastebin service [Электронный ресурс]. – Режим доступа: <https://paste.rs/>. – Дата доступа: 12.02.2026.
- [4] Advanced Bash-Scripting Guide [Электронный ресурс]. – Режим доступа: <https://tldp.org/LDP/abs/html/>. – Дата доступа: 12.02.2026.
- [5] Shell Scripting Tutorial [Электронный ресурс]. – Режим доступа: <https://www.shellscript.sh/>. – Дата доступа: 12.02.2026.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг программного кода

Листинг А.1 – Код программы

```
#!/usr/bin/env bash
# Simple paste client using https://paste.rs
# Supports creating a paste from stdin or file, optional language/type

set -euo pipefail

usage() {
    cat <<EOF
Usage: $0 [options] [id|file]
Create new paste from stdin or file, or download by id.

Options:
  -f FILE      read input from FILE (upload)
  -l LANG      language for highlighting (metadata). Auto-detected from
file ext if omitted
  -t TYPE      type: text|code (default: text). If code and LANG given
it'll wrap in fenced block
  -o FILE      output downloaded paste into FILE (download)
  -h           show this help

Examples:
  echo testing | $0 -t text
  $0 -f script.py -t code      # upload file
  $0 QEXe0syg                  # download paste with id
EOF
}

detect_lang() {
    local fname="$1"
    case "${fname##*.}" in
        py) echo python ;;
        sh) echo bash ;;
        js) echo javascript ;;
        ts) echo typescript ;;
        c) echo c ;;
        cpp | cc | cxx | hpp) echo cpp ;;
        java) echo java ;;
        rb) echo ruby ;;
        php) echo php ;;
        html | htm) echo html ;;
        css) echo css ;;
        json) echo json ;;
        go) echo go ;;
        rs) echo rust ;;
        lua) echo lua ;;
        md) echo markdown ;;
        *) echo "" ;;
    esac
}
```



```

TYPE=text
LANG=""
INFILE=""
OUTFILE=""

while getopts ":f:l:t:o:h" opt; do
    case $opt in
        f) INFILE="$OPTARG" ;;
        l) LANG="$OPTARG" ;;
        t) TYPE="$OPTARG" ;;
        o) OUTFILE="$OPTARG" ;;
        h)
            usage
            exit 0
            ;;
        \?)
            echo "Unknown option: -$OPTARG" >&2
            usage
            exit 2
            ;;
    esac
done
shift $((OPTIND - 1))

# If a positional argument remains and -f not used, it may be an
existing file (upload) or an id (download)
if [ -n "${1-}" ]; then
    if [ -z "$INFILE" ] && [ -f "$1" ]; then
        INFILE="$1"
    else
        ID_OR_ARG="$1"
    fi
fi

# If downloading: ID provided (or URL)
if [ -n "${ID_OR_ARG-}" ] && [ -z "$INFILE" ]; then
    # treat as id or url
    if [ "${ID_OR_ARG}" =~ ^https?:// ]; then
        URL="$ID_OR_ARG"
    else
        URL="https://paste.rs/$ID_OR_ARG"
    fi
    if [ -n "$OUTFILE" ]; then
        # download to temp, optionally strip fenced code blocks if
        outfile extension matches
        TMPD=$(mktemp)
        if ! curl -sSf "$URL" -o "$TMPD"; then
            echo "Download failed" >&2
            rm -f "$TMPD"
            exit 1
        fi
        OUTLANG=$(detect_lang "$OUTFILE")
        # read first and last lines to detect fenced code block
        firstline=$(sed -n '1p' "$TMPD" || true)
        stripped=0
        # Match fence with language: ``lang

```

```

        if echo "$firstline" | grep -E -q
'^[[:space:]]*````[[:space:]]*$'; then
            # fence without lang
            if [ -n "$OUTLANG" ]; then
                sed '1d;$d' "$TMPD" > "$OUTFILE"
                stripped=1
            fi
        else
            if echo "$firstline" | grep -E -q
'^[[:space:]]*````[[:alnum:]]+_.-+[[:space:]]*$'; then
                fenced_lang=$(echo "$firstline" | sed -E 's/
^[[:space:]]*````([[:alnum:]]+_.-+)[[:space:]]*$/\1/')
                if [ -n "$OUTLANG" ] && [ "$OUTLANG" =
"$fenced_lang" ]; then
                    sed '1d;$d' "$TMPD" > "$OUTFILE"
                    stripped=1
                fi
            fi
        fi
        if [ $stripped -eq 0 ]; then
            mv "$TMPD" "$OUTFILE"
        else
            rm -f "$TMPD"
        fi
    else
        curl -sSf "$URL"
    fi
    exit $?
fi

# Create new paste
# determine language if not set and file provided
if [ -z "$LANG" ] && [ -n "$INFILE" ]; then
    LANG=$(detect_lang "$INFILE")
fi

# Prepare content (possibly wrapped) into temp file then send
TMP=$(mktemp)
trap 'rm -f "$TMP"' EXIT

if [ -n "$INFILE" ]; then
    if [ "$TYPE" = "code" ] && [ -n "$LANG" ]; then
        printf '````%s\n' "$LANG" > "$TMP"
        cat "$INFILE" >> "$TMP"
        printf '\n```\n' >> "$TMP"
    else
        cat "$INFILE" > "$TMP"
    fi
else
    # read from stdin
    if [ "$TYPE" = "code" ] && [ -n "$LANG" ]; then
        printf '````%s\n' "$LANG" > "$TMP"
        cat - >> "$TMP"
        printf '\n```\n' >> "$TMP"
    else
        cat - > "$TMP"
    fi
fi

```

```
fi

# send to paste.rs
RESP=$(curl -sS --data-binary "@${TMP}" https://paste.rs/)
if [ -z "$RESP" ]; then
    echo "Upload failed or empty response" >&2
    exit 1
fi

# paste.rs returns a URL like https://paste.rs/<id>
# print id to stdout
ID=$(basename "$RESP")
echo "$ID"

exit 0
```